Assignment 5: DevOps and CI/CD for ReviewGuard

Theoretical Research and Practical Implementation Report

Dias Kazikhanov

Part 1. Theoretical report

Case study: ReviewGuard, a research system for sentiment and authenticity analysis of online reviews

1. DevOps as a way of working

DevOps brings software development and operations into one feedback loop. Developers, researchers, and the people who run a system share responsibility for changes from source code through testing, packaging, release, and monitoring. Its purpose is not simply to install more tools. It is to make small, reviewable changes safe to integrate and straightforward to reproduce. Common practices include version control, automated builds, continuous testing, infrastructure described as code, clear ownership, and learning from failures.

These practices address a familiar research problem: a result can depend on code, data, configuration, software versions, hardware, and random choices that are not recorded together. When those pieces live in different places or are prepared by hand, another researcher may be unable to reconstruct the experiment. DevOps makes the software process visible and repeatable. It does not decide whether a research question is sound or a conclusion is justified; it helps ensure the computational procedure is inspectable and can be rerun.

The benefits for a research group are practical. Version control records which code produced an analysis. Automated checks catch broken data-processing assumptions earlier. A container or environment specification reduces differences between a laptop and a server. A pipeline makes the same checks run for every proposed change. Together these practices reduce setup time, prevent regressions, and make handoffs between researchers easier. Wilson and colleagues recommend treating scientific software as part of the research apparatus and emphasize version control, testing, automation, and code review as ways to improve the reliability of computational work (Wilson et al., 2014a).

2. Continuous integration and delivery

Continuous integration (CI) means integrating small code changes frequently and automatically checking each change with a repeatable build and test suite. A commit should trigger fast feedback. If a check fails, the team can identify the change while it is still easy to understand. The DORA CI guidance describes automated builds and tests for each commit as core practices and recommends keeping the shared code base in a working state (DORA, n.d.-a).

Continuous delivery (CD) extends that path by keeping a tested version ready to release and automating the steps needed to deliver it to a test or production environment. Continuous deployment is a stricter form in which every qualifying change is deployed automatically. These terms are related but not identical: a research team may use continuous delivery while keeping a human approval before publishing a model or exposing an API to users (DORA, n.d.-b).

For research software, a useful pipeline can lint or validate inputs, install a declared environment, run unit and integration tests, build a container image, and optionally deploy it to an isolated test service. Each run should identify the source revision and preserve relevant outputs such as test reports, package versions, configuration, and experiment logs. The pipeline should test software behavior, not claim that a model result is scientifically valid merely because the build passed.

3. Why automation matters in research

Research code changes as hypotheses, datasets, and analysis plans evolve. A small edit to normalization, label mapping, a split rule, or a metric can change the interpretation of an experiment. Automated tests can protect stable expectations: schema validity, deterministic splitting, input limits, API response shape, and metric calculations. Osborne and colleagues specifically recommend automated tests linked to version control for computational research; tests can compare code against analytical expectations, prototypes, or published values where suitable (Osborne et al., 2014).

For ReviewGuard, a Python research project that studies sentiment and review authenticity, CI is useful at two levels. Unit tests can check that data adapters preserve required fields and that an inference response is well formed. A container build can verify that the API and its declared runtime dependencies are compatible. The existing GitHub Actions workflow already runs the project test suite on pushes and pull requests. Jenkins provides a second, self-hostable route that executes the repository's `Jenkinsfile`, tests a Python environment, builds a Docker image, and checks the running API's health endpoint.

Reproducibility still requires more than a passing test suite. Research teams should record dataset versions and provenance, preprocessing choices, model and tokenizer versions, random seeds, hardware/runtime information, and the exact source revision. Large datasets and trained checkpoints should be managed as explicit inputs or artifacts rather than silently baked into an image. Where exact dependency resolution matters, a lock file and pinned image digest are stronger than broad minimum-version requirements. The project can improve reproducibility in stages without claiming bitwise-identical results across all hardware.

4. Tools and their role in the research environment

Docker packages an application with its runtime and dependencies as an image. A research API can then start in a controlled environment on a laptop, CI runner, or test server. Docker's Python guide describes the image as a portable unit containing the application, dependencies, configuration, and runtime (Docker, n.d.). For ReviewGuard, the `Dockerfile` builds a Python 3.12 API image. It excludes local corpora and model weights from the build context so that data remain separately managed; a trained checkpoint can be mounted into the container. Docker reduces environment drift, but it does not by itself freeze data, model weights, seeds, hardware behavior, or package resolution.

Jenkins is an automation server that runs build and deployment workflows. A `Jenkinsfile` stores those steps beside the source code. Pipeline as Code makes the definition reviewable and versioned, and Jenkins can discover or run it for repository branches (Jenkins, n.d.-a; Jenkins, n.d.-b). In this assignment, stages install research dependencies and run pytest, build the API image, and launch a short-lived container for a health check. This arrangement is useful when a lab needs a private runner or wants to connect tests to local compute resources. Jenkins needs maintenance, credentials, and a carefully scoped build environment, so it is not automatically simpler than a hosted CI service.

Kubernetes manages containerized applications across a cluster using declarative configuration and automation (Kubernetes, n.d.). A research group could use it to run a prediction API, scheduled preprocessing jobs, or resource-intensive training tasks on shared infrastructure. Jobs and manifests can record resource requests, environment variables, and mounted artifacts. Kubernetes is appropriate when multiple services, concurrent workloads, or cluster scheduling justify its operational cost. A small research prototype such as ReviewGuard does not need Kubernetes to achieve a repeatable Docker build and CI pipeline; it remains an optional deployment and scaling layer.

Together the tools form one possible workflow: Git stores code and the `Jenkinsfile`; Jenkins checks a change; Docker packages the tested API; Kubernetes can deploy that image when a cluster is

useful. In a different environment, GitHub Actions or GitLab CI can perform the integration and build steps without changing the core idea. The tool should match the group's infrastructure and capacity to maintain it.

5. Conclusion

DevOps and CI/CD solve coordination and repeatability problems that often surround scientific software. They automate routine checks, make changes traceable, and shorten the time between an edit and evidence that something broke. For a research team, those improvements make computational methods easier to inspect, share, rerun, and maintain. Docker helps control the application environment; Jenkins or a hosted CI service runs the checks; Kubernetes can manage deployment at larger scale.

The key limit is that operational reliability is not scientific validity. A green pipeline cannot establish that a dataset is representative, that labels are defensible, or that a model supports a broad empirical claim. It can establish that a specified revision passed specified tests and produced a runnable artifact. When teams preserve that distinction and record data, configurations, dependencies, and experiment provenance alongside the code, DevOps becomes a practical part of reproducible research rather than an end in itself.

References

DORA. (n.d.-a). *Continuous integration*. https://dora.dev/capabilities/continuous-integration/

DORA. (n.d.-b). *Continuous delivery*. https://dora.dev/capabilities/continuous-delivery/

Docker. (n.d.). *Python language-specific guide*. https://docs.docker.com/guides/python/

Jenkins. (n.d.-a). *Pipeline*. https://www.jenkins.io/doc/book/pipeline/

Jenkins. (n.d.-b). *Pipeline as Code*. https://www.jenkins.io/doc/book/pipeline/pipeline-as-code/

Kubernetes. (n.d.). *Concepts*. https://kubernetes.io/docs/concepts/

Osborne, J. M., Bernabeu, M. O., Bruna, M., Calderhead, B., Cooper, J., Dalchau, D., et al. (2014). Ten simple rules for effective computational research. *PLoS Computational Biology, 10*(1), e1 003 506. https://doi.org/10.1371/journal.pcbi.1003506

Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., et al. (2014a). Best practices for scientific computing. *PLoS Biology, 12*(1), e1 001 745. https://doi.org/10.1371/journal.pbio.1001745

Part 2. Practical report

Project and repository

ReviewGuard is a Python research project for sentiment and authenticity analysis of online reviews. The project repository is github.com/Svoppy/review-api (https://github.com/Svoppy/review-api). The root `Jenkinsfile` defines the CI stages, and the root `Dockerfile` defines the API image.

Implementation

The Docker image uses Python 3.12, installs the project as a package, runs Uvicorn as a non-root user, and exposes port 8000. Inference dependencies are included by default. Local datasets, model weights, generated reports, and development files are excluded from the build context; a trained checkpoint is mounted separately at `/models/latest`. The `INSTALL_INFERENCE=0` build argument produces a smaller image for testing API startup and `/health` without downloading the large model stack.

The local Jenkins server is defined in `devops/jenkins/compose.yml` and built from `devops/jenkins/Dockerfile`. Jenkins loads the pipeline from the repository's `Jenkinsfile`. Its stages install the declared research and development dependencies in Python 3.12, run `PYTHONPATH=src python -m pytest -q`, build the Docker image, and start a temporary container to check `/health`. Docker Hub publishing and Kubernetes deployment are not enabled; credentials and a target cluster are not configured.

The repository already had a GitHub Actions workflow that runs pytest on pushes and pull requests. The Jenkins setup adds the requested self-hosted workflow rather than replacing the existing CI. The local demonstration used a temporary Git snapshot of the current working tree because the Assignment 5 files are not yet published to the public remote. The Jenkins job itself is configured to load its pipeline definition from SCM.

Pipeline evidence

Jenkins build **#10** completed successfully from revision `e9e91b1` of the local SCM snapshot. All **152 tests passed in 3.64 seconds**; Jenkins then built `reviewguard-ci:10`, started a temporary container, and received a successful response from `/health`. The accompanying Jenkins Stage View screenshot records the green result. The local server is available at `http://127.0.0.1:8080` while its Docker Compose service is running.

Difficulties and conclusions

The machine did not have a running Docker daemon at first, so a local Docker runtime was started. Jenkins also needed its Docker CLI, Git in the Python test agent, packaged FastAPI static assets, and a writable local home directory. The initial health check exposed that the API package omitted its HTML file; adding it to package data fixed the container startup. The CPU-only runner also uses the CPU PyTorch wheel so CI does not download unused CUDA libraries. The Docker socket and Jenkins endpoint are kept local for this demonstration; the Compose file should be adapted to the organization's security and infrastructure before a shared deployment.

The run demonstrates that repository code can be tested, packaged, and started through one Jenkins pipeline. The health check confirms that the API process starts; it does not establish model accuracy or scientific validity. For a shared runner, the next steps are to publish the assignment changes, connect Jenkins to the public GitHub repository with webhooks, pin dependency/image versions, and store model artifacts with explicit provenance. The existing GitHub Actions workflow continues to run the full test suite on pushes and pull requests.